UNIVERSIDAD CATÓLICA BOLIVIANA

INFORME
SEGUNDA EVALUACIÓN

ASIGNATURA:
INTERNET DE LAS COSAS

INTEGRANTES
ALAN ARIEL MALDONADO CARVAJAL
ELIAS MILAN LECOÑA CONDORI
JUAN JOSE CLAROS SUNTURA

DOCENTE:
ING. PAMELA VALENZUELA

CARRERA: INGENIERIA DE SISTEMAS

FECHA DE ENTREGA: 14 / 10 / 2026

LA PAZ – BOLIVIA

2026

AQUACONTROL: SISTEMA INTELIGENTE DE MEDICIÓN Y GESTIÓN DEL CONSUMO DE AGUA

1. Introducción

1.1. Descripción general del proyecto y su importancia

AquaControl es un sistema IoT cliente-servidor para medir, visualizar y controlar el consumo de agua en un domicilio. Un ESP32 programado en MicroPython lee dos sensores, el caudalímetro YF-S201 y el sensor de agua T1592, y maneja dos actuadores: una electroválvula que corta el paso del agua y un LED RGB que indica el consumo. Las lecturas se guardan en una base de datos PostgreSQL alojada en Supabase. Un servidor Node.js publicado en Internet con ngrok entrega el dashboard web: cualquier persona con el enlace puede registrarse, vincular su dispositivo y ver en tiempo real el estado de los cuatro componentes. Un superadministrador puede consultar los usuarios registrados en el sistema.

La importancia del proyecto está en convertir el consumo de agua, que normalmente solo se conoce a fin de mes por la factura, en información inmediata y accionable. Saber cuántos litros se usan, cuándo y a qué ritmo permite detectar desperdicios y fugas a tiempo, fijar metas de ahorro y reaccionar de forma automática ante situaciones de riesgo como una inundación.

Este informe corresponde a la segunda fase del proyecto. En la primera fase se construyó la base del sistema (autenticación, API REST, base de datos y dashboard) con series matemáticas simuladas como fuente de datos. En esta fase las series se reemplazan por datos reales de sensores, se incorporan los actuadores, la base de datos pasa a la nube (Supabase), el sistema se publica con ngrok y el ESP32 se vincula a la cuenta del usuario sin escribir claves en el hardware.

1.2. Planteamiento del problema

En los hogares no existe una forma sencilla de saber cuánta agua se consume en tiempo real. Según la presentación de la propuesta, alrededor del 30 % del agua potable se desperdicia sin un control efectivo: grifos mal cerrados, duchas prolongadas, fugas en las instalaciones que pasan desapercibidas durante días. El medidor de la empresa distribuidora solo informa el total acumulado, no permite fijar límites ni actúa por sí mismo cuando ocurre un problema.

Pregunta central: ¿Cómo puede un usuario conocer y controlar su consumo de agua en tiempo real desde cualquier lugar, y cómo puede la propia instalación protegerse cuando se supera un consumo definido o se produce una fuga?

1.3. Objetivos

Objetivo general

Desarrollar un sistema IoT cliente-servidor basado en ESP32 que mida el consumo de agua en tiempo real, almacene los datos en Supabase, los muestre en un dashboard web accesible desde Internet y controle automáticamente una electroválvula ante un límite de consumo o la detección de una fuga.

Objetivos específicos

- Medir caudal (L/min) y volumen (L) con el sensor YF-S201 y detectar agua en el piso con el sensor analógico T1592.
- Controlar una electroválvula NC de 220 VAC mediante un módulo relé, de forma automática (fuga o límite) y remota (dashboard).
- Indicar el consumo con un LED RGB que se enciende de forma gradual y cambia de verde a amarillo y a rojo según el caudal.
- Vincular el ESP32 a la cuenta del usuario de forma automática, sin copiar claves al código del dispositivo.
- Almacenar los datos en PostgreSQL (Supabase) con integridad referencial y políticas de seguridad por fila (RLS).
- Publicar el servidor con ngrok para que cualquier persona con el enlace pueda acceder al sistema.
- Implementar registro, inicio de sesión y roles (usuario y superadministrador) con contraseñas cifradas y JWT.
- Construir un dashboard que muestre en tiempo real los dos sensores y los dos actuadores.

1.4. Alcance del proyecto

- **Hardware:** un punto de medición (tubería de 1/2") con ESP32, YF-S201, T1592, módulo relé, electroválvula NC y LED RGB, más su modelo de simulación en Wokwi.
- **Firmware:** MicroPython con portal de configuración Wi-Fi, vinculación automática, lectura de sensores, lógica local de seguridad y envío de lecturas.
- **Nube de datos:** Supabase (PostgreSQL) con las tablas cliente, dispositivo, lectura_caudal y evento, funciones para el ESP32 y RLS.
- **Servidor:** Node.js + Express publicado con ngrok; sirve el dashboard y la API REST de la aplicación.
- **Cliente:** dashboard web responsivo (React + Vite + Recharts) con vistas de usuario y de superadministrador.

Quedan fuera de esta fase la aplicación móvil nativa (Flutter) planteada en la propuesta inicial, que se reemplaza por el dashboard web responsivo, y la clasificación automática del consumo por actividad (ducha, lavadora, riego). El servidor corre en una PC del equipo: el sitio está disponible mientras esa PC y ngrok estén encendidos, aunque las lecturas del ESP32 se siguen guardando en Supabase.

2. Diseño del Sistema IoT

2.1. Arquitectura general del sistema

El sistema sigue el modelo cliente-servidor sobre una arquitectura IoT en capas (Figura 1):

- **Nodo IoT (percepción, actuación y borde):** el ESP32 lee los sensores cada segundo, decide localmente si cierra la válvula y qué color muestra el LED, y cada 5 segundos envía un resumen a Supabase.
- **Nube de datos (Supabase):** guarda las lecturas. El ESP32 solo puede ejecutar dos funciones (anunciar_dispositivo y registrar_lecturas); todas las tablas están protegidas con RLS.
- **Servidor (Node.js + Express):** corre en una PC del equipo; sirve el dashboard, expone la API REST de la aplicación (usuarios, dispositivos, consultas, administración) y reenvía las lecturas nuevas a los navegadores en tiempo real.
- **ngrok:** crea un túnel HTTPS desde Internet hasta el puerto 3000 de esa PC, con un dominio fijo del tipo aquacontrol.ngrok-free.app.
- **Clientes:** navegadores de usuarios y del superadministrador, desde cualquier red.

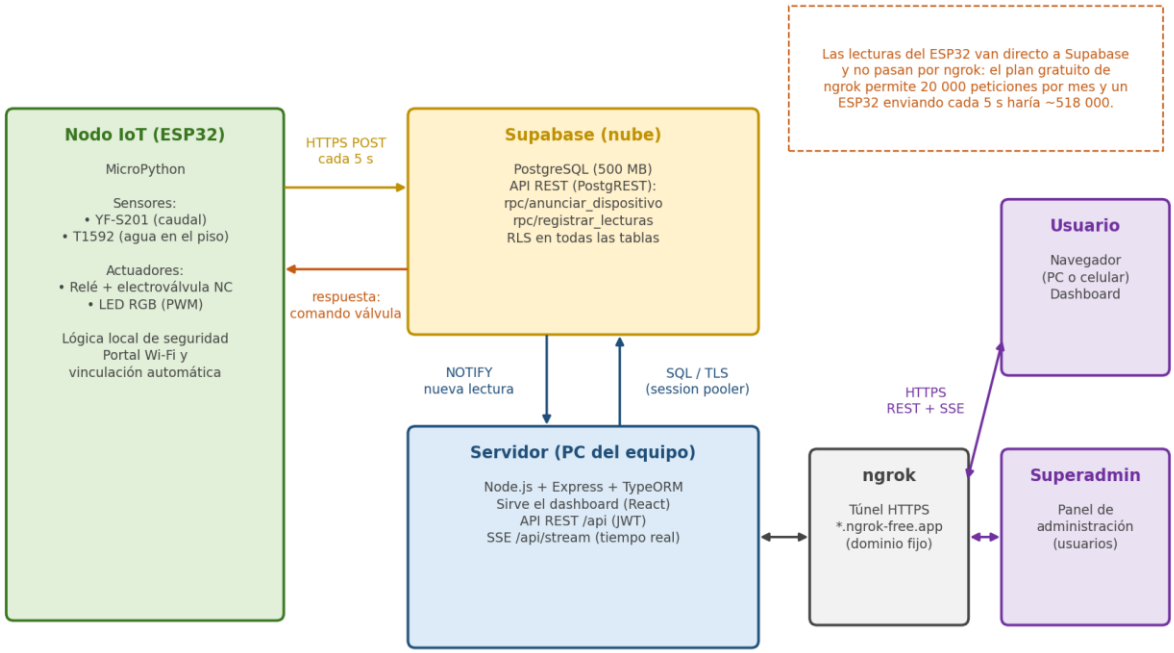


Figura 1. Arquitectura cliente-servidor de AquaControl.

Una decisión clave fue que el ESP32 no envíe sus datos a través de ngrok. El plan gratuito de ngrok permite 20 000 peticiones HTTP por mes y 1 GB de transferencia; un ESP32 que envía cada 5 segundos haría unas 518 000 peticiones al mes, y un dashboard que consulta cada pocos segundos consumiría el resto. Supabase, en cambio, no limita la cantidad de peticiones a su API en el plan gratuito. Por eso las lecturas van directo a Supabase y ngrok solo

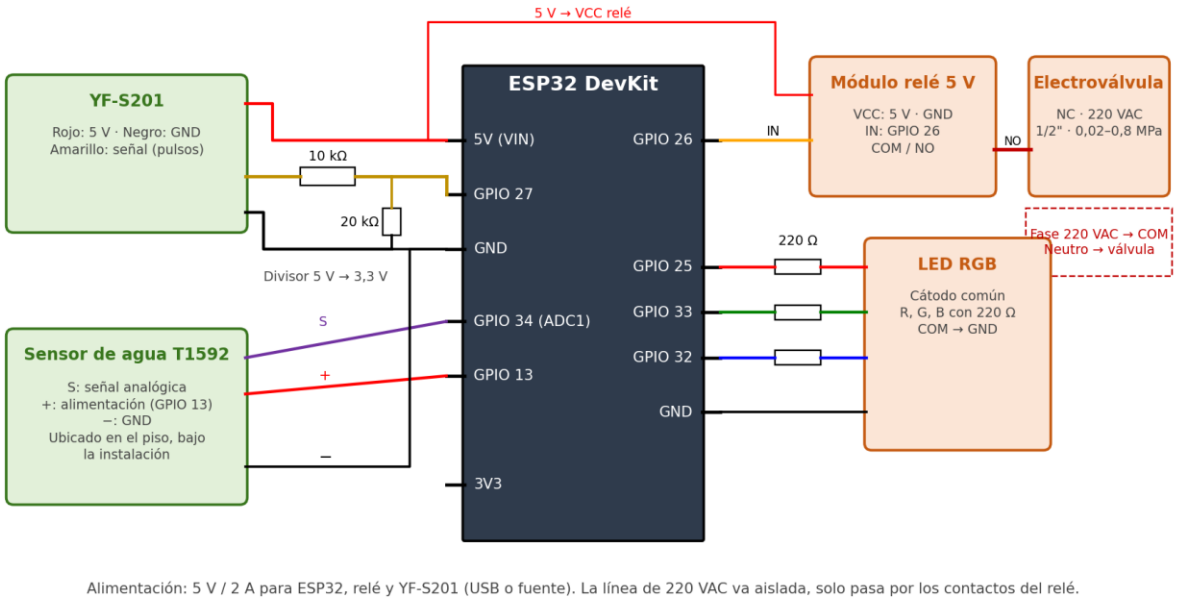
transporta el tráfico de los navegadores, que además usa una sola conexión persistente (SSE) por pestaña abierta para el tiempo real.

Tabla 1. Límites de los servicios gratuitos usados.

Servicio (plan gratuito)	Límite	Consumo previsto de AquaControl
ngrok	20 000 peticiones/mes, 1 GB/mes, 1 dominio fijo	Carga de páginas, acciones del usuario y una conexión SSE por pestaña
Supabase	500 MB de base de datos, 5 GB de egreso, peticiones ilimitadas	17 280 lecturas/día por dispositivo (una cada 5 s), ≈ 2,5 MB/día
Supabase (pausa)	El proyecto se pausa tras 7 días sin actividad	El ESP32 encendido mantiene el proyecto activo

2.2. Diagrama del circuito y red

Circuito



Alimentación: 5 V / 2 A para ESP32, relé y YF-S201 (USB o fuente). La línea de 220 VAC va aislada, solo pasa por los contactos del relé.

Figura 2. Diagrama de conexiones del nodo AquaControl.

Tabla 2. Tabla de conexiones.

Componente	Pin del componente	Conexión en el ESP32	Observación
YF-S201	Rojo / Negro / Amarillo	5V (VIN) / GND / GPIO 27	Divisor 10 kΩ – 20 kΩ en la señal (5 V → 3,3 V)
Sensor T1592	S / + / −	GPIO 34 / GPIO 13 / GND	GPIO 34 es ADC1 (compatible con Wi-Fi); se alimenta solo al medir

Módulo relé	VCC / GND / IN	5V / GND / GPIO 26	Contactos COM–NO en serie con la fase de 220 VAC
Electroválvula NC	Bobina 220 VAC	Contacto NO del relé y neutro	Relé activado = válvula abierta
LED RGB (cátodo común)	R / G / B / COM	GPIO 25 / 33 / 32 / GND	Resistencia de 220 Ω en cada color, PWM a 1 kHz
Botón BOOT	—	GPIO 0	Mantenerlo 5 s borra el Wi-Fi y la vinculación

Se eligió una electroválvula normalmente cerrada (NC): si se corta la energía o el ESP32 se reinicia, la válvula se cierra por sí sola, un comportamiento a prueba de fallas (fail-safe) adecuado para evitar inundaciones. Como contrapartida, la bobina permanece energizada mientras la válvula está abierta, por lo que debe ser de servicio continuo. La parte de 220 VAC se monta en una caja aislada y solo pasa por los contactos del relé. La válvula requiere una presión mínima de 0,02 MPa, que la red domiciliaria cumple.

Modelo de simulación en Wokwi

El circuito se modeló y probó en el simulador Wokwi (ESP32 con MicroPython). Como Wokwi no tiene sensor de flujo ni electroválvula, se usaron equivalencias: un potenciómetro deslizante fija el caudal y el propio ESP32 genera los pulsos con PWM en GPIO 4, puenteado a GPIO 27 (f = 7,5 × Q, igual que el YF-S201); un LED azul conectado al contacto NO del relé representa la electroválvula, y un potenciómetro giratorio alimentado desde GPIO 13 reemplaza al T1592. En la simulación se verificó el cálculo del caudal (≈ 10 L/min), el cambio de color del LED, el cierre de la válvula al detectar agua y su reapertura cuando el sensor vuelve a estar seco. Los archivos están en la carpeta Fase2/wokwi del repositorio.

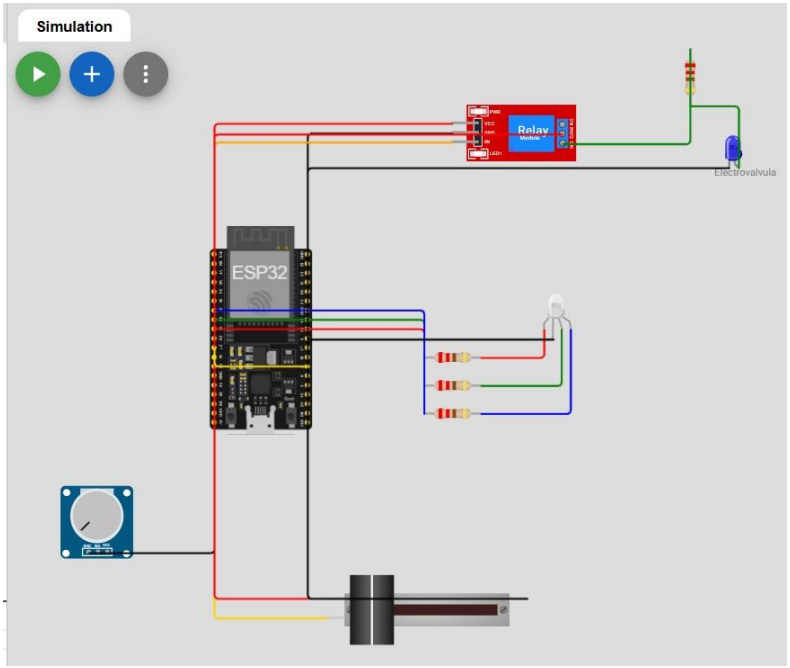


Figura 3. Simulación del circuito en Wokwi

Red

- **ESP32 → Supabase:** el ESP32 se conecta como estación (STA) al Wi-Fi del domicilio (2,4 GHz) y envía HTTPS a https://<proyecto>.supabase.co. No depende de que el servidor esté encendido.
- **Servidor → Supabase:** el back-end se conecta a PostgreSQL por el session pooler de Supabase (puerto 5432, TLS), que funciona en redes IPv4; la conexión directa del plan gratuito es solo IPv6.
- **Navegador → ngrok → servidor:** el usuario abre https://aquacontrol.ngrok-free.app; ngrok reenvía el tráfico por un túnel cifrado al puerto 3000 de la PC. No se abren puertos en el router.

```
# En la PC del servidor (una sola vez):
ngrok config add-authtoken <token de la cuenta de ngrok>
# Cada vez que se levanta el sistema:
npm run start                                  # back-end en el puerto 3000
ngrok http 3000 --url https://aquacontrol.ngrok-free.app
```

En el plan gratuito, la primera visita desde un navegador muestra una página de aviso de ngrok; al pulsar "Visit Site" no vuelve a aparecer durante 7 días en ese navegador.

2.3. Protocolos utilizados

Tabla 3. Protocolos del sistema.

Capa	Protocolo / estándar	Uso en AquaControl
Enlace	Wi-Fi IEEE 802.11 b/g/n (2,4 GHz), WPA2	ESP32 en modo estación; en modo punto de acceso para el portal de configuración
Red / transporte	IPv4, TCP, TLS 1.2/1.3	Todas las conexiones hacia Internet van cifradas (HTTPS)
Aplicación	HTTP/1.1 REST	API de Supabase (PostgREST) para el ESP32 y API de Express para el dashboard
Tiempo real	Server-Sent Events (SSE)	El servidor envía cada lectura nueva al navegador por una conexión abierta
Túnel	ngrok (HTTPS)	Publica el servidor local en un dominio público fijo
Base de datos	Protocolo PostgreSQL + LISTEN/NOTIFY	Consultas del back-end y aviso de lecturas nuevas
Formato y seguridad	JSON, JWT (RFC 7519), SHA-256, bcrypt	Cuerpo de los mensajes, sesiones, tokens de dispositivo y contraseñas
Hardware	Interrupciones GPIO, ADC de 12 bits, PWM (LEDC)	Pulsos del YF-S201, lectura del T1592 y control del LED RGB

Se eligió HTTP/REST en lugar de MQTT porque Supabase ya ofrece una API REST con autenticación y RLS, no requiere instalar ni publicar un broker, y el envío cada 5 segundos cubre la frecuencia necesaria.

2.4. Descripción de componentes y sensores

Tabla 4. Componentes del sistema.

Componente	Tipo	Características principales	Función

ESP32 DevKit	Microcontrolador	Doble núcleo 240 MHz, Wi-Fi 2,4 GHz, ADC 12 bits, 3,3 V	Procesa, decide y comunica
YF-S201	Sensor digital (efecto Hall)	1–30 L/min, 5–18 V, f (Hz) = 7,5 × Q (L/min), ≈ 450 pulsos/L, máx. 1,75 MPa	Mide caudal y volumen
T1592	Sensor analógico de agua	3–5 V, < 20 mA, salida proporcional al área mojada, área ≈ 40 × 16 mm	Detecta agua en el piso (fugas)
Electroválvula	Actuador	NC, 220 VAC, rosca 1/2", 0,02–0,8 MPa	Abre o corta el paso de agua
Módulo relé	Interfaz de potencia	Bobina 5 V, contactos 10 A / 250 VAC, optoacoplado	Conmuta los 220 VAC de la válvula
LED RGB	Actuador visual	Cátodo común, controlado por PWM (brillo 0–100 %)	Indica el caudal y las alertas

Sensor de flujo YF-S201

Tiene un rotor con un imán que gira con el paso del agua; un sensor de efecto Hall genera un pulso por cada vuelta. El ESP32 cuenta los pulsos con una interrupción por flanco de bajada y, cada segundo, calcula el caudal como Q = (pulsos / Δt) / 7,5 y el volumen como pulsos acumulados / 450. El factor de pulsos por litro se puede ajustar desde el dashboard para calibrar cada dispositivo.

Sensor de agua T1592: ubicación y uso

El T1592 es una placa con pistas expuestas que entrega un voltaje mayor cuanto más superficie está mojada. No sirve para medir el agua dentro de la tubería, pero es ideal como detector de fugas. Se coloca en el piso, en el punto más bajo bajo la instalación (debajo del lavaplatos, junto al caudalímetro y la electroválvula), con las pistas hacia arriba, para que el agua derramada lo cubra.

- **Detección de fugas:** si la lectura supera el umbral (37 % de la escala), el ESP32 cierra la válvula de inmediato, sin esperar al servidor, y registra un evento FUGA.
- **Reapertura controlada:** la válvula no se reabre sola: el usuario confirma desde el dashboard que revisó la fuga.
- **Protección del sensor:** se alimenta desde GPIO 13 solo durante la medición (unos 10 ms por segundo) para reducir la corrosión de las pistas.

Como complemento, el YF-S201 permite detectar fugas internas (grifo goteando, inodoro que pierde agua): si el caudal nunca vuelve a cero durante un tiempo prolongado, se registra un evento FLUJO_CONTINUO.

Electroválvula y relé

La válvula se abre solo si se cumplen tres condiciones: no hay fuga, el consumo del día no superó el límite y el usuario no la cerró desde el dashboard. El módulo relé aísla el ESP32 de los 220 VAC.

LED RGB

Tabla 5. Estados del LED RGB.

Situación	Comportamiento del LED

Sin consumo	Apagado
Caudal bajo (0–4 L/min)	Verde; el brillo sube de forma gradual con el caudal
Caudal medio (4–8 L/min)	Transición de verde a amarillo
Caudal alto (> 8 L/min)	Rojo con brillo máximo ("pasa mucha agua")
Límite diario alcanzado	Rojo que se enciende y apaga suavemente (respiración)
Fuga detectada	Parpadeo alternado rojo / azul
Esperando vinculación	Azul parpadeando lento

El encendido gradual se logra con PWM: cada 20 ms el brillo se acerca un 6 % hacia el valor objetivo, lo que produce una transición suave en lugar de un salto.

3. Diseño de la Base de Datos

3.1. Tipo de base de datos y justificación

Se usa PostgreSQL administrado por Supabase. Se eligió porque:

- Los datos tienen una estructura fija y relaciones claras (un cliente tiene dispositivos; un dispositivo tiene lecturas y eventos), que se protegen con claves foráneas.
- Las consultas principales son por rango de fechas y agregaciones (promedio, máximo, suma de pulsos), que SQL resuelve con índices compuestos.
- Supabase aloja la base en la nube sin costo, ofrece copias de seguridad, un editor SQL y una API REST lista para usar, y aplica Row Level Security (RLS) a esa API.
- Es PostgreSQL estándar: el back-end de la fase 1 (TypeORM) se conecta cambiando solo la cadena de conexión.

La propuesta inicial mencionaba Firebase. Se descartó porque su modelo documental dificulta las agregaciones por rango de tiempo y porque el equipo ya contaba con el modelo relacional de la fase 1.

3.2. Estructura de tablas y campos

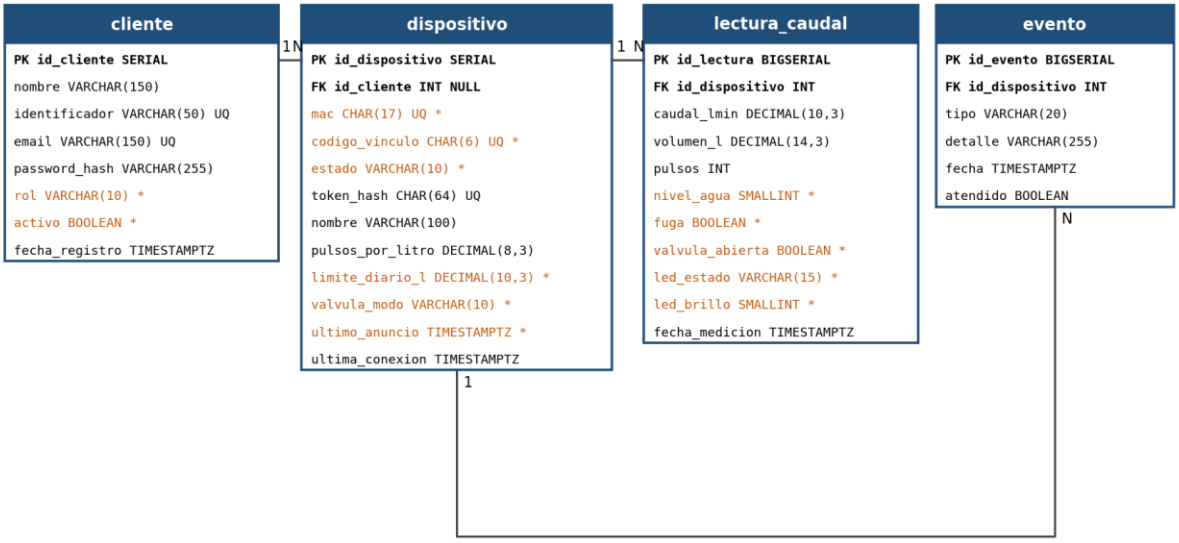


Figura 4. Modelo entidad-relación.

Tabla 6. Tablas de la base de datos.

Tabla	Propósito	Campos destacados
cliente	Usuarios del sistema	email e identificador únicos, password_hash (bcrypt), rol (usuario o admin), activo
dispositivo	Cada ESP32; existe desde que se anuncia, aunque aún no tenga dueño	mac, codigo_vinculo, estado, token_hash, limite_diario_l, valvula_modo, ultimo_anuncio
lectura_caudal	Una fila cada 5 s con el estado de los 4 componentes	caudal_lmin, volumen_l, pulsos, nivel_agua, fuga, valvula_abierta, led_estado, led_brillo
evento	Alertas y acciones relevantes	tipo (FUGA, LIMITE, FLUJO_CONTINUO, VALVULA), detalle, atendido

Cada fila de lectura_caudal guarda también el estado de los actuadores (válvula abierta o cerrada, color y brillo del LED). Así la electroválvula y el LED tienen su propio historial y se pueden graficar igual que un sensor.

3.3. Restricciones y tipos de datos

- **Claves primarias:** SERIAL para cliente y dispositivo; BIGSERIAL para lectura_caudal y evento, que son las tablas que más crecen.
- **Claves foráneas:** lectura_caudal y evento → dispositivo con ON DELETE CASCADE. dispositivo.id_cliente admite NULL mientras el equipo está pendiente de vinculación.
- **Unicidad:** email e identificador del cliente; mac, codigo_vinculo y token_hash del dispositivo.
- **Precisión:** DECIMAL para caudal y volumen, evitando los errores de redondeo de los tipos flotantes.

- **CHECK:** valores no negativos, límite mayor que cero, nivel_agua entre 0 y 4095, led_brillo entre 0 y 100 y listas cerradas para rol, estado, valvula_modo, led_estado y tipo de evento.
- **Índices:** (id_dispositivo, fecha_medicion DESC) para las consultas por rango e (id_dispositivo, fecha DESC) para los eventos.

```
CREATE TABLE cliente (
  id_cliente     SERIAL PRIMARY KEY,
  nombre         VARCHAR(150) NOT NULL,
  identificador  VARCHAR(50)  NOT NULL UNIQUE,
  email          VARCHAR(150) NOT NULL UNIQUE,
  password_hash  VARCHAR(255) NOT NULL,
  rol            VARCHAR(10)  NOT NULL DEFAULT 'usuario' CHECK (rol IN
('usuario','admin')),
  activo         BOOLEAN      NOT NULL DEFAULT TRUE,
  fecha_registro TIMESTAMPTZ  NOT NULL DEFAULT NOW());

CREATE TABLE dispositivo (
  id_dispositivo   SERIAL PRIMARY KEY,
  id_cliente       INT REFERENCES cliente(id_cliente) ON DELETE SET NULL,
  mac              CHAR(17)  NOT NULL UNIQUE,
  codigo_vinculo   CHAR(6)   NOT NULL UNIQUE,
  estado           VARCHAR(10) NOT NULL DEFAULT 'pendiente' CHECK (estado IN
('pendiente','vinculado')),
  token_hash       CHAR(64) UNIQUE,
  nombre           VARCHAR(100),
  pulsos_por_litro DECIMAL(8,3)  NOT NULL DEFAULT 450 CHECK (pulsos_por_litro > 0),
  limite_diario_l  DECIMAL(10,3) CHECK (limite_diario_l > 0),
  valvula_modo     VARCHAR(10) NOT NULL DEFAULT 'auto' CHECK (valvula_modo IN
('auto','abierta','cerrada')),
  ultimo_anuncio   TIMESTAMPTZ,
  ultima_conexion  TIMESTAMPTZ);

CREATE TABLE lectura_caudal (
  id_lectura      BIGSERIAL PRIMARY KEY,
  id_dispositivo  INT NOT NULL REFERENCES dispositivo(id_dispositivo) ON DELETE CASCADE,
  caudal_lmin     DECIMAL(10,3) NOT NULL CHECK (caudal_lmin >= 0),
  volumen_l       DECIMAL(14,3) NOT NULL CHECK (volumen_l >= 0),
  pulsos          INT NOT NULL CHECK (pulsos >= 0),
  nivel_agua      SMALLINT CHECK (nivel_agua BETWEEN 0 AND 4095),
  fuga            BOOLEAN NOT NULL DEFAULT FALSE,
  valvula_abierta BOOLEAN NOT NULL,
  led_estado      VARCHAR(15) CHECK (led_estado IN ('apagado','verde','amarillo','rojo',
                                                    'limite','fuga','vinculando')),
  led_brillo      SMALLINT CHECK (led_brillo BETWEEN 0 AND 100),
  fecha_medicion  TIMESTAMPTZ NOT NULL);

CREATE TABLE evento (
  id_evento      BIGSERIAL PRIMARY KEY,
  id_dispositivo INT NOT NULL REFERENCES dispositivo(id_dispositivo) ON DELETE CASCADE,
  tipo           VARCHAR(20) NOT NULL CHECK (tipo IN
('FUGA','LIMITE','FLUJO_CONTINUO','VALVULA')),
  detalle        VARCHAR(255),
  fecha          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  atendido       BOOLEAN NOT NULL DEFAULT FALSE);

CREATE INDEX idx_lectura_disp_fecha ON lectura_caudal(id_dispositivo, fecha_medicion
DESC);
CREATE INDEX idx_evento_disp_fecha  ON evento(id_dispositivo, fecha DESC);
```

3.4. Seguridad (RLS y políticas)

Supabase publica automáticamente una API REST sobre las tablas, accesible con la clave pública (anon). Por eso la seguridad se diseña en tres niveles:

- **RLS activado en todas las tablas:** el rol anon (la API pública, que usa el ESP32) no tiene políticas sobre ninguna tabla, por lo que no puede leer ni modificar filas directamente.
- **Funciones como única puerta del ESP32:** anon solo puede ejecutar anunciar_dispositivo y registrar_lecturas. Son funciones SECURITY DEFINER que validan la MAC o el token antes de tocar las tablas.
- **Políticas por usuario para el servidor:** el back-end se conecta con un rol propio (aquacontrol_api) sujeto a RLS y, en cada transacción, fija el cliente y el rol del token JWT. Las políticas solo devuelven sus filas; el superadmin ve todas.

```
ALTER TABLE cliente        ENABLE ROW LEVEL SECURITY;
ALTER TABLE dispositivo    ENABLE ROW LEVEL SECURITY;
ALTER TABLE lectura_caudal ENABLE ROW LEVEL SECURITY;
ALTER TABLE evento         ENABLE ROW LEVEL SECURITY;

-- La API pública (anon) no ve las tablas: solo ejecuta las dos funciones del ESP32
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM anon, authenticated;
GRANT EXECUTE ON FUNCTION anunciar_dispositivo(text), registrar_lecturas(text, jsonb) TO
anon;

-- El servidor fija por transacción: SET LOCAL app.id_cliente = '7'; SET LOCAL app.rol =
'usuario';
CREATE FUNCTION es_admin() RETURNS boolean LANGUAGE sql STABLE AS
  $$ SELECT current_setting('app.rol', true) = 'admin' $$;
CREATE FUNCTION cliente_actual() RETURNS int LANGUAGE sql STABLE AS
  $$ SELECT nullif(current_setting('app.id_cliente', true), '')::int $$;

CREATE POLICY dispositivo_propio ON dispositivo TO aquacontrol_api
  USING (id_cliente = cliente_actual() OR es_admin());
CREATE POLICY lectura_propia ON lectura_caudal FOR SELECT TO aquacontrol_api
  USING (es_admin() OR id_dispositivo IN
         (SELECT id_dispositivo FROM dispositivo WHERE id_cliente = cliente_actual()));
CREATE POLICY evento_propio ON evento TO aquacontrol_api
  USING (es_admin() OR id_dispositivo IN
         (SELECT id_dispositivo FROM dispositivo WHERE id_cliente = cliente_actual()));
CREATE POLICY cliente_servidor ON cliente TO aquacontrol_api
  USING (true);   -- login y registro; el filtrado por rol lo hace la API
```

- **Contraseñas:** bcrypt con 10 rondas de sal; la columna password_hash nunca se devuelve en las respuestas.
- **Tokens de dispositivo:** 24 bytes aleatorios generados por la base de datos; solo se guarda su hash SHA-256 y el token se entrega al ESP32 una única vez.
- **Secretos fuera del código:** la cadena de conexión de Supabase, el secreto JWT y el authtoken de ngrok están en el archivo .env del servidor, que no se sube al repositorio.

4. Integración mediante API REST

4.1. Uso de API REST

El sistema usa dos API REST con responsabilidades separadas:

- **API de Supabase (PostgREST), usada por el ESP32:** dos funciones expuestas como POST /rest/v1/rpc/<función>. El ESP32 envía la clave pública del proyecto (la misma para todos los dispositivos, incluida en el firmware) y su token personal dentro del cuerpo.
- **API de Express, usada por el dashboard:** en https://aquacontrol.ngrok-free.app/api, construida en capas (rutas → controladores → servicios → repositorios TypeORM) y autenticada con Authorization: Bearer <JWT>.

Tabla 7. Endpoints del sistema.

Método	Ruta	Acceso	Descripción
POST	/rest/v1/rpc/anunciar_dispositivo	ESP32	Registra la MAC; devuelve el código o el token
POST	/rest/v1/rpc/registrar_lecturas	ESP32	Guarda lecturas; devuelve el comando de la válvula
POST	/api/auth/registro · /api/auth/login	Público	Crea cuenta o inicia sesión; devuelve JWT
GET	/api/clientes/me	Usuario	Perfil del usuario autenticado
POST	/api/dispositivos/vincular	Usuario	Vincula un ESP32 con su código de 6 caracteres
GET	/api/dispositivos	Usuario	Lista sus dispositivos con el estado en línea
PATCH	/api/dispositivos/:id	Usuario	Nombre, pulsos por litro o límite diario
POST	/api/dispositivos/:id/valvula	Usuario	Modo de la válvula: auto, abierta o cerrada
DELETE	/api/dispositivos/:id	Usuario	Desvincula el dispositivo (vuelve a pendiente)
GET	/api/dispositivos/:id/lecturas	Usuario	Lecturas del rango (desde, hasta, agrupar_s)
GET	/api/dispositivos/:id/resumen	Usuario	Promedio, máximo y litros del rango
GET	/api/dispositivos/:id/eventos	Usuario	Alertas registradas
GET	/api/stream?dispositivo=:id	Usuario	Canal SSE con las lecturas en tiempo real
GET	/api/admin/usuarios	Superadmin	Usuarios registrados con sus dispositivos
PATCH	/api/admin/usuarios/:id	Superadmin	Activar o suspender una cuenta

4.2. Inserción de datos (POST)

El ESP32 mide cada segundo y, cada 5 segundos, envía una lectura que resume ese intervalo (caudal promedio, pulsos sumados y el último estado de los actuadores). Como no tiene reloj de tiempo real, indica hace cuántos milisegundos se midió (ms_atras) y la función calcula la fecha exacta.

```
POST https://<proyecto>.supabase.co/rest/v1/rpc/registrar_lecturas
apikey: <clave pública del proyecto>
Content-Type: application/json

{ "p_token": "9b1f...c07e",
  "p_lecturas": [
    { "caudal_lmin": 6.42, "volumen_l": 12.875, "pulsos": 241, "nivel_agua": 210,
      "fuga": false, "valvula_abierta": true, "led_estado": "amarillo",
      "led_brillo": 62, "ms_atras": 0 } ] }

200 OK
{ "guardadas": 1, "valvula": "abrir", "limite_diario_l": 250, "consumo_hoy_l": 145.3 }
```

La función valida el token, revisa que cada valor esté dentro de su rango (las restricciones CHECK son la última barrera), inserta las filas, actualiza ultima_conexion, registra eventos de fuga o límite y avisa al servidor con NOTIFY para que la lectura aparezca en los dashboards abiertos.

4.3. Lectura de datos (GET)

El dashboard consulta la API de Express. Para rangos largos, el parámetro agrupar_s hace que el servidor promedie las lecturas por intervalos, de modo que la gráfica nunca recibe más de unos 800 puntos.

```
GET /api/dispositivos/3/resumen?desde=2026-10-14T00:00:00Z&hasta=2026-10-14T23:59:59Z
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

200 OK
{ "cantidad_lecturas": 8624, "caudal_promedio": 0.412, "caudal_maximo": 11.83,
  "litros_consumidos": 145.3, "minutos_valvula_cerrada": 34, "fugas": 1 }
```

Los litros consumidos se calculan con la suma de pulsos dividida por pulsos_por_litro, y no con la diferencia del volumen acumulado, porque ese acumulado vuelve a cero cada vez que el ESP32 se reinicia.

4.4. Manejo de errores

En Express, los servicios lanzan una excepción AppError con el código HTTP y un mensaje claro; un middleware central la convierte en { "error": "mensaje" }. Los errores no previstos se registran en el servidor y se responden con 500 sin exponer detalles. En Supabase, las funciones usan RAISE EXCEPTION con un código propio, que PostgREST devuelve como 400.

Tabla 8. Códigos de error.

Código	Cuándo ocurre	Reacción del cliente
400 Bad Request	Datos inválidos o token del ESP32 rechazado por la función	El dashboard muestra el mensaje; el ESP32 descarta el token y vuelve a anunciarse
401 Unauthorized	JWT ausente, inválido o vencido	El dashboard cierra la sesión y lleva al login
403 Forbidden	Un usuario sin rol admin entra a /api/admin o la cuenta está suspendida	Mensaje de acceso denegado
404 Not Found	El dispositivo no existe, es de otro usuario o el código no se anunció	Mensaje en pantalla

409 Conflict	Email o identificador ya registrado; dispositivo ya vinculado	Mensaje en el formulario
5xx	Supabase o el servidor no responden	El ESP32 conserva las lecturas en cola y reintenta

5. Desarrollo en MicroPython

El objetivo de esta fase es que el ESP32 se instale sin editar código. El mismo main.py se graba una sola vez con Thonny en todos los dispositivos; el Wi-Fi y la vinculación con la cuenta se hacen desde el celular y el dashboard.

5.1. Configuración de red

Al encender, el ESP32 busca el archivo wifi.json. Si no existe o no logra conectarse en 20 segundos, crea su propia red Wi-Fi AquaControl-XXXXXX (las últimas 6 cifras de su MAC) y levanta un pequeño servidor web en 192.168.4.1. El usuario se conecta con el celular, elige su red, escribe la contraseña y el ESP32 la guarda y se reinicia. Es el mismo mecanismo que usan los enchufes y focos inteligentes comerciales. Mantener presionado el botón BOOT 5 segundos borra la configuración.

```
def conectar_o_portal():
    datos = leer_json("wifi.json")
    if datos and conectar(datos["ssid"], datos["clave"], espera_s=20):
        return
    ap = network.WLAN(network.AP_IF)
    ap.config(essid="AquaControl-" + CODIGO)      # CODIGO = últimas 6 cifras de la MAC
    ap.active(True)
    ssid, clave = portal_web("192.168.4.1")       # formulario HTML con las redes
encontradas
    guardar_json("wifi.json", {"ssid": ssid, "clave": clave})
    machine.reset()
```

Una vez configurado, la conexión no bloquea la medición: si se pierde el Wi-Fi, el ESP32 sigue midiendo y controlando la válvula, y reintenta la conexión cada 10 segundos.

5.2. Vinculación automática con la cuenta del usuario

En lugar de copiar una clave API al código, el ESP32 se identifica con su MAC, que es única de fábrica, y recibe su token automáticamente (Figura 5).

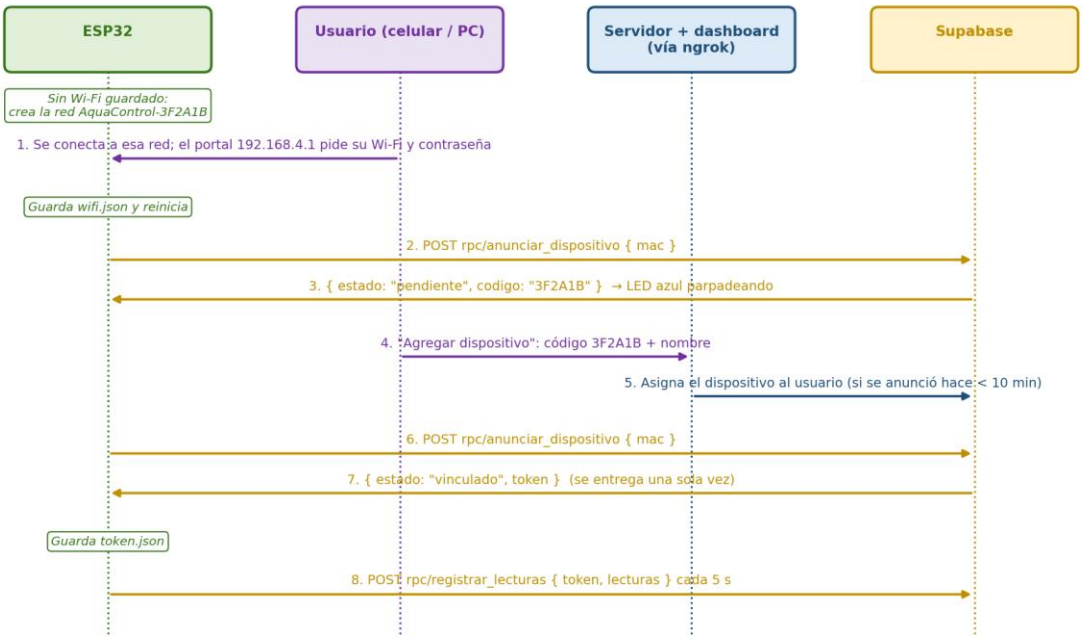


Figura 5. Secuencia de configuración y vinculación del ESP32.

- El ESP32 llama a anunciar_dispositivo con su MAC cada 10 segundos. Supabase crea el dispositivo en estado pendiente con un código de 6 caracteres (las últimas cifras de la MAC). El LED parpadea en azul y el código aparece en la consola de Thonny y en una etiqueta pegada al equipo.
- El usuario, ya con sesión iniciada, pulsa "Agregar dispositivo", escribe ese código y un nombre. El servidor solo acepta códigos que se anunciaron en los últimos 10 minutos y que no tienen dueño.
- En el siguiente anuncio, la función genera un token aleatorio, guarda solo su hash y lo devuelve una única vez. El ESP32 lo guarda en token.json y empieza a enviar lecturas.
- Si el usuario desvincula el equipo desde el dashboard, el token deja de valer; el ESP32 lo borra y vuelve a anunciarse.

La única clave que lleva el firmware es la clave pública (anon) del proyecto de Supabase, igual para todos los dispositivos. Supabase la diseñó para incluirse en aplicaciones cliente: con RLS activado solo permite ejecutar las dos funciones anteriores.

5.3. Lectura de datos (mínimo tres variables)

Cada segundo el firmware obtiene las variables de los dos sensores y el estado de los dos actuadores: caudal (L/min), volumen (L) y pulsos del YF-S201; nivel de agua (0–4095) y bandera de fuga del T1592; estado de la válvula; y color y brillo del LED.

```
def contar_pulso(pin):                # interrupción del YF-S201
    global pulsos
    pulsos += 1
sensor = Pin(27, Pin.IN, Pin.PULL_UP)
sensor.irq(trigger=Pin.IRQ_FALLING, handler=contar_pulso)
```

```
def leer_nivel_agua():                # T1592, alimentado solo al medir
    agua_alim.value(1); time.sleep_ms(10)
    nivel = sum(agua_adc.read() for _ in range(5)) // 5
    agua_alim.value(0)
    return nivel

estado = machine.disable_irq(); n = pulsos; pulsos = 0; machine.enable_irq(estado)
caudal  = (n / dt) / 7.5              # L/min
volumen = total_pulsos / 450          # L
nivel   = leer_nivel_agua()
fuga    = nivel >= UMBRAL_FUGA
abrir   = not fuga and consumo_hoy < limite and modo != "cerrada"
mover_valvula(abrir)
led_estado, led_brillo = actualizar_led(caudal, fuga, limite_alcanzado)
```

5.4. Envío de datos

Cada 5 segundos se arma una lectura con los promedios del intervalo y se agrega a una cola. Si hay Wi-Fi, se envía con HTTPS a registrar_lecturas; la respuesta trae el modo de la válvula y el límite configurados en el dashboard, que el ESP32 aplica en ese momento.

```
cuerpo = {"p_token": token, "p_lecturas": cola[:20]}
r = requests.post(SUPABASE_URL + "/rest/v1/rpc/registrar_lecturas",
                  data=json.dumps(cuerpo),
                  headers={"apikey": SUPABASE_ANON, "Content-Type": "application/json"})
if r.status_code == 200:
    del cola[:len(cuerpo["p_lecturas"])]   # solo se borra lo que Supabase confirmó
    resp = r.json()
    modo, limite = resp["valvula"], resp["limite_diario_l"]
```

5.5. Manejo de errores y reconexión

- **Sin Wi-Fi o sin Internet:** las lecturas se acumulan en la cola (hasta 120, unos 10 minutos) y se envían al volver la conexión, con su hora real gracias a ms_atras.
- **Errores HTTP:** la cola solo se vacía con una respuesta 200; ante un error 5xx o un timeout se reintenta en el siguiente ciclo. Si Supabase rechaza el token (400), el ESP32 lo borra y vuelve al modo de vinculación.
- **Excepciones y memoria:** cada envío está protegido con try/except/finally; la respuesta se cierra siempre y se llama a gc.collect(), porque una conexión TLS ocupa unos 40 KB de RAM.
- **Seguridad sin red:** el cierre por fuga y por límite se decide en el ESP32, así que funciona aunque Supabase o el Wi-Fi estén caídos.
- **Bloqueos:** un watchdog (machine.WDT de 30 s) reinicia el ESP32 si el programa se cuelga; al reiniciar, la válvula NC queda cerrada hasta evaluar el estado.

6. Sistema de Autenticación

6.1. Registro e inicio de sesión

El registro pide nombre, identificador, email y contraseña. El servidor valida que estén completos, que el email tenga formato válido y que la contraseña tenga al menos 6 caracteres;

comprueba que el email y el identificador no estén en uso (409) y guarda la contraseña cifrada con bcrypt. Toda cuenta nueva tiene el rol usuario. El inicio de sesión compara la contraseña con el hash y, ante cualquier fallo, responde el mismo mensaje genérico "Credenciales inválidas" para no revelar si el email existe. Una cuenta suspendida por el superadmin no puede iniciar sesión.

[Insertar captura: Pantallas de inicio de sesión y registro]

Figura 6. Pantallas de inicio de sesión y registro

6.2. Gestión de sesiones

Las sesiones no se guardan en el servidor: el token JWT, firmado con un secreto del servidor, contiene el id del cliente y su rol, y vence a las 8 horas. El front-end lo guarda en localStorage y lo agrega como Authorization: Bearer en cada petición. Al recargar la página, el contexto de autenticación recupera el perfil con GET /api/clientes/me; si el token venció, se elimina y se redirige al login. Las rutas privadas están protegidas por un componente ProtectedRoute, y las de administración por uno que además exige el rol admin.

6.3. Roles y superadministrador

El sistema tiene dos roles. El rol usuario solo ve y controla sus dispositivos. El rol admin (superadmin) accede además a un panel con la lista de usuarios registrados: nombre, email, fecha de registro, cantidad de dispositivos y última conexión, con la opción de suspender o reactivar cuentas. El superadmin no ve las contraseñas ni puede mover válvulas ajenas.

El rol admin no se puede obtener desde la página de registro. El primer superadmin se asigna directamente en el editor SQL de Supabase:

```
UPDATE cliente SET rol = 'admin' WHERE email = 'admin@aquacontrol.bo';
```

[Insertar captura: Panel de superadministrador con los usuarios registrados]

Figura 7. Panel de superadministrador con los usuarios registrados

6.4. Seguridad de acceso

- Cada usuario solo ve y controla sus dispositivos: la API verifica la propiedad en cada petición y la base de datos lo refuerza con RLS.
- El middleware soloAdmin comprueba el rol del token en todas las rutas /api/admin.
- Usuarios y dispositivos tienen credenciales separadas: el token de un ESP32 solo sirve para enviar lecturas de ese equipo; no permite leer datos ni mover válvulas.
- Todo el tráfico desde Internet va cifrado: ngrok y Supabase solo aceptan HTTPS.
- Como mejora se propone limitar los intentos de inicio de sesión por IP, ya que el sitio es público.

7. Dashboard en Tiempo Real

7.1. Diseño del dashboard

El dashboard es una aplicación web en React (Vite), responsiva, servida por el mismo servidor Express a través de ngrok. Una barra lateral lista los dispositivos del usuario con un punto verde o gris según estén en línea. Al elegir uno se muestra una vista con un panel por componente (Figura 8): dos sensores y dos actuadores.



Figura 8. Diseño del dashboard: un panel por componente.

Aunque la electroválvula y el LED no son sensores, también generan datos: su estado en cada momento. Por eso cada uno tiene su propio panel con el estado actual, su historial y, en el caso de la válvula, los controles. El LED se presenta como un gemelo digital, una réplica en pantalla del color y brillo que tiene el LED físico.

Tabla 9. Paneles del dashboard por componente.

Panel	Indicador principal	Gráfica en tiempo real	Interacción
Sensor de flujo YF-S201	Caudal actual (L/min) y consumo del día respecto al límite	Línea de caudal y área de volumen	Definir el límite diario
Sensor de agua T1592	Medidor del % mojado y estado SECO / FUGA	Línea del nivel con la línea del umbral	Confirmar que se revisó una fuga

Electroválvula	ABIERTA / CERRADA y motivo (manual, límite o fuga)	Gráfica escalonada abierta–cerrada	Botones Abrir, Cerrar y Automático
LED RGB	Círculo con el color y el brillo actuales	Barras de brillo coloreadas por estado	Solo visualización

7.2. Visualización de datos

Se usan gráficas de Recharts sobre un eje de tiempo común, para poder relacionar los cuatro paneles: en la Figura 8 se ve cómo el aumento del nivel de agua (fuga) coincide con el cierre de la válvula y con el LED en azul. Cuando el dispositivo estuvo desconectado, las líneas se cortan en lugar de unir los extremos. Los rangos disponibles son 10 minutos, 1 hora, 24 horas y uno personalizado; en los rangos largos se usan datos promediados por el servidor.

7.3. Actualización en tiempo real

Al abrir un dispositivo, el navegador abre una conexión Server-Sent Events con GET /api/stream. Cuando Supabase guarda una lectura, la función registrar_lecturas ejecuta NOTIFY; el servidor, que está escuchando con LISTEN, reenvía la lectura solo a los navegadores del dueño de ese dispositivo, y los cuatro paneles se actualizan sin recargar. El retardo total, desde la medición hasta la pantalla, es de 5 a 6 segundos.

Esta técnica usa una sola petición HTTP por pestaña abierta, lo que es clave para no agotar el límite de ngrok. Si la conexión se corta, el navegador la reabre solo y pide las lecturas que faltaron desde la última recibida. Un dispositivo se considera en línea si envió datos en los últimos 15 segundos.

7.4. Interactividad

- Cambiar el rango de tiempo de las gráficas o elegir fechas exactas, y exportar las lecturas a CSV.
- Definir el límite diario y abrir, cerrar o dejar en automático la electroválvula (la orden llega al ESP32 en menos de 5 s).
- Confirmar una alerta de fuga para permitir que la válvula se vuelva a abrir.
- Vincular dispositivos con su código, renombrarlos, ajustar los pulsos por litro o desvincularlos.
- Superadmin: buscar usuarios, ver sus dispositivos y suspender o reactivar cuentas.

7.5. Manejo de errores

- Los mensajes de error de la API se muestran tal como los envía el servidor; si no hay mensaje, se usa uno genérico.
- Si el dispositivo deja de enviar datos, su estado pasa a "Desconectado" y se indica hace cuánto llegó la última lectura.
- Si la conexión SSE se corta, se muestra "Reconectando…" y se reintenta automáticamente.
- Un código de vinculación inválido o vencido muestra cómo reiniciar el anuncio del ESP32.
- Un error 401 (sesión vencida) cierra la sesión; un 403 en el panel de administración muestra "Acceso denegado".

8. Conclusiones

8.1. Resultados obtenidos

- Se migró el sistema de la fase 1 (series simuladas) a datos reales de sensores, conservando la autenticación y la arquitectura en capas.
- Se diseñó una arquitectura cliente-servidor publicada con ngrok y con los datos en Supabase, compatible con los límites de los planes gratuitos.
- El ESP32 se configura y vincula sin editar código: Wi-Fi desde un portal en el celular y vinculación con un código de 6 caracteres.
- El dashboard presenta los dos sensores y los dos actuadores en tiempo real, y el superadmin puede consultar los usuarios registrados.
- El circuito completo se modeló y verificó en Wokwi: el caudal simulado se calcula correctamente, el LED cambia de color con el caudal y la válvula se cierra al detectar agua.

8.2. Problemas y soluciones

Tabla 10. Problemas encontrados y soluciones.

Problema	Solución
ngrok gratuito: 20 000 peticiones/mes, insuficiente para un ESP32 enviando cada 5 s	El ESP32 envía a Supabase; ngrok solo atiende navegadores y el tiempo real usa SSE (1 petición por pestaña)
Copiar la clave API a config.py en cada ESP32 era propenso a errores	Vinculación automática por MAC con código de 6 caracteres y token entregado una sola vez
Escribir el Wi-Fi en el código obligaba a usar Thonny en cada cambio de red	Portal de configuración en modo punto de acceso
La conexión directa a Supabase es solo IPv6	Uso del session pooler (IPv4) en el puerto 5432
La API pública de Supabase expone las tablas	RLS en todas las tablas y acceso del ESP32 solo mediante funciones
500 MB de base de datos en el plan gratuito	Una lectura cada 5 s (promedio del intervalo) en lugar de una por segundo
La señal del YF-S201 es de 5 V y el ESP32 trabaja a 3,3 V	Divisor resistivo de 10 kΩ y 20 kΩ
El ADC2 del ESP32 no funciona con Wi-Fi y las pistas del T1592 se corroen	T1592 en GPIO 34 (ADC1), alimentado solo al medir
Wokwi no incluye sensor de flujo ni electroválvula	Pulsos generados por PWM y LED en el contacto del relé

8.3. Aprendizaje logrado

El proyecto permitió integrar todas las capas de un sistema IoT: la electrónica (adaptación de niveles, aislamiento de 220 VAC, interrupciones, ADC y PWM), la programación de un microcontrolador con recursos limitados, el diseño de una base de datos en la nube con control de acceso por fila y la publicación de un servidor local en Internet. También dejó dos lecciones de diseño: los servicios gratuitos tienen límites que condicionan la arquitectura y hay que calcularlos antes de elegir, y en IoT la red no es confiable, por lo que las decisiones críticas, como cerrar el agua ante una fuga, deben tomarse en el propio dispositivo.

9. Anexos

9.1. Código fuente (GitHub)

Repositorio: https://github.com/AlanMC7650/IoTProyecto

Carpeta	Contenido
Back/	Servidor Node.js (Express, TypeORM): API REST, SSE y archivos del dashboard
Front/	Dashboard web (React, Vite, Recharts)
ESP32/	Firmware MicroPython: main.py, portal Wi-Fi y vinculación
supabase/	Script SQL: tablas, funciones, RLS y políticas
Fase2/wokwi/	Modelo de simulación: diagram.json, main.py y guía de uso

9.2. Capturas

[Insertar captura: Montaje físico del circuito]

Figura 9. Montaje físico del circuito

[Insertar captura: Portal de configuración Wi-Fi en el celular]

Figura 10. Portal de configuración Wi-Fi en el celular

[Insertar captura: Vinculación del dispositivo con su código]

Figura 11. Vinculación del dispositivo con su código

[Insertar captura: Dashboard con los cuatro paneles en tiempo real]

Figura 12. Dashboard con los cuatro paneles en tiempo real

[Insertar captura: Tablas en el editor de Supabase]

Figura 13. Tablas en el editor de Supabase

10. Referencias

Espressif Systems. (2023). ESP32 Series Datasheet. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

MicroPython. (2024). Quick reference for the ESP32. https://docs.micropython.org/en/latest/esp32/quickref.html

Seeed Studio. (s.f.). G1/2 Water Flow Sensor (YF-S201). https://wiki.seeedstudio.com/G1_2_Water_Flow_sensor/

Supabase. (2025). Connecting to your database. https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase. (2025). Row Level Security. https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase. (2025). Pricing (límites del plan gratuito). https://supabase.com/pricing

ngrok. (2025). Free plan limits. https://ngrok.com/docs/pricing-limits/free-plan-limits/

PostgreSQL Global Development Group. (2025). Row Security Policies y NOTIFY. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

WHATWG. (2025). HTML Living Standard: Server-sent events. https://html.spec.whatwg.org/multipage/server-sent-events.html

OpenJS Foundation. (2025). Express – Manejo de errores. https://expressjs.com/es/guide/error-handling.html

Meta Platforms. (2025). React – Documentación. https://react.dev/

Recharts. (2025). Recharts API. https://recharts.org/en-US/api

Jones, M., Bradley, J. y Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). IETF. https://www.rfc-editor.org/rfc/rfc7519

OWASP Foundation. (2021). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Wokwi. (2025). Wokwi Docs. https://docs.wokwi.com/